

Autómatas

Tema 3 Gramáticas



Introducción

- **Sintaxis.**

(Del lat. *syntaxis*, y este del gr. σύνταξις, de συντάσσειν, coordinar).

1. f. *Gram.* Parte de la gramática que enseña a coordinar y unir las palabras para formar las oraciones y expresar conceptos.
2. f. *Inform.* Conjunto de reglas que definen las secuencias correctas de los elementos de un lenguaje de programación.

Diccionario de la RAE



Conceptos de Gramáticas, I

- Equivalencia de gramáticas:
 - G_1 y G_2 son equivalentes si $L(G_1)=L(G_2)$
- Forma Normal de Chomsky
 - $\mathcal{P}=\{ (A \rightarrow BC) \cup (S \rightarrow \lambda) \cup (A \rightarrow a) \mid A,B,C \in \Sigma_N, a \in \Sigma_T$
- Forma Normal de Greibach
 - $\mathcal{P}=\{ (A \rightarrow aX) \cup (S \rightarrow \lambda) \cup (A \rightarrow a) \mid A \in \Sigma_N, X \in \Sigma_N^*, a \in \Sigma_T$

Conceptos de Gramáticas, II

- Gramáticas bien formadas
 - Limpias:
 - Sin reglas innecesarias:
 - $A \rightarrow A$
 - Sin símbolos inaccesibles:
 - $A \in \Sigma_N \mid \sim \exists (B \rightarrow xAy) \in \wp$
 - Sin símbolos superfluos:
 - $\sim \exists A \rightarrow_* x \mid x \in \Sigma_T^*$
 - Sin reglas no generativas:
 - $A \rightarrow \lambda \wedge A \neq S$
 - Sin reglas de redenominación:
 - $A \rightarrow B \mid A, B \in \Sigma_N$

Conceptos de Gramáticas, III

- Algoritmos de limpieza de GIC
 - Eliminación de Reglas innecesarias:
 - Sea una GIC G y $\wp$ contiene producciones $\wp_i$ de la forma $A \rightarrow A$, entonces $\wp' = \wp - \wp_i$
 - Eliminación de Símbolos inaccesibles y superfluos:
 - Sea la GIC $G = (\Sigma_T, \Sigma_N, S, \wp)$ y la G transformada $G' = (\Sigma'_T, \Sigma'_N, S, \wp')$
 1. $\Sigma'_N = \{S\}, \Sigma'_T = \emptyset, \wp' = \emptyset$
 2. Repetir
 - $\forall A \in \Sigma'_N$ y $\forall A \rightarrow w \in \wp$
 - 2.1. $\wp' = \wp' \cup \{A \rightarrow w\}$
 - 2.2. $\forall B \in w, \Sigma'_N = \Sigma'_N \cup \{B\}$
 - 2.3. $\forall a \in w, \Sigma'_T = \Sigma'_T \cup \{a\}$Hasta que no se puedan añadir nuevas reglas a $\wp'$

Conceptos de Gramáticas, IV

- Algoritmos de limpieza de GIC
 - Eliminación de Reglas no generativas:
 - Sea la GIC $G = (\Sigma_T, \Sigma_N, S, \wp)$ y la transformada $G' = (\Sigma_T, \Sigma_N, S, \wp')$
 - 1. Obtención de símbolos anulables Σ_A
 - 1.1. $\Sigma_A = \{A \mid A \in \Sigma_N \text{ y } A \rightarrow \lambda \in \wp\}$
 - 1.2. Repetir $\forall B \mid B \rightarrow w \in \wp, w \in \Sigma_N^*$
Si $w \in \Sigma_A^* \Rightarrow \Sigma_A = \Sigma_A \cup \{w\}$
Hasta que no se añadan más símbolos no terminales a Σ_A
 - 2. Creación de G'
 - 2.1. $\wp' = \emptyset$
 - 2.2. $\forall B \rightarrow x_1 x_2 \dots x_n \in \wp, x_1 x_2 \dots x_n \in \Sigma^*$ hacer todas las $B \rightarrow y_1 y_2 \dots y_n$ tal que:
 - Si $x_i \notin \Sigma_A \Rightarrow y_i = x_i$
 - Si $x_i \in \Sigma_A \Rightarrow y_i = x_i \text{ o } \lambda$
 - 2.3. Aplicar la propiedad de concatenación para eliminar λ y eliminar las reglas $B \rightarrow \lambda$ las reglas resultantes se incluyen en $\wp'$
 - 2.4. Si $S \in \Sigma_A \Rightarrow \wp' = \wp' \cup \{S \rightarrow \lambda\}$

Conceptos de Gramáticas, V

- Algoritmos de limpieza de GIC
 - Eliminación de Reglas no generativas:
Ejemplo

Si $B \rightarrow x_1 x_2 x_3 \ \forall \ x_2, x_3 \in \Sigma_A$

Entonces hacer las producciones:

$B \rightarrow x_1 x_2 x_3 \mid x_1 x_2 \lambda \mid x_1 \lambda x_3 \mid x_1 \lambda \lambda$

Entonces $\wp' = \wp' \cup \{B \rightarrow x_1 x_2 x_3 \mid x_1 x_2 \mid x_1 x_3 \mid x_1\}$

Conceptos de Gramáticas, VI

- Algoritmos de limpieza de GIC
 - Eliminación de Reglas de Redenominación (no funciona si hay reglas $A \rightarrow B \rightarrow \dots \rightarrow A$):
 - Sea la GIC $G = (\Sigma_T, \Sigma_N, S, \wp)$ y la transformada $G' = (\Sigma_T, \Sigma_N, S, \wp')$

Repetir:

Para cada regla unitaria $A \rightarrow B$

Sean $B \rightarrow w_1 | w_1 | \dots | w_n$

Entonces $\wp' = \wp - \{A \rightarrow B\} \cup \{A \rightarrow w_1 | w_1 | \dots | w_n\}$

Hasta que no haya más reglas unitarias

Conceptos de Gramáticas, VII

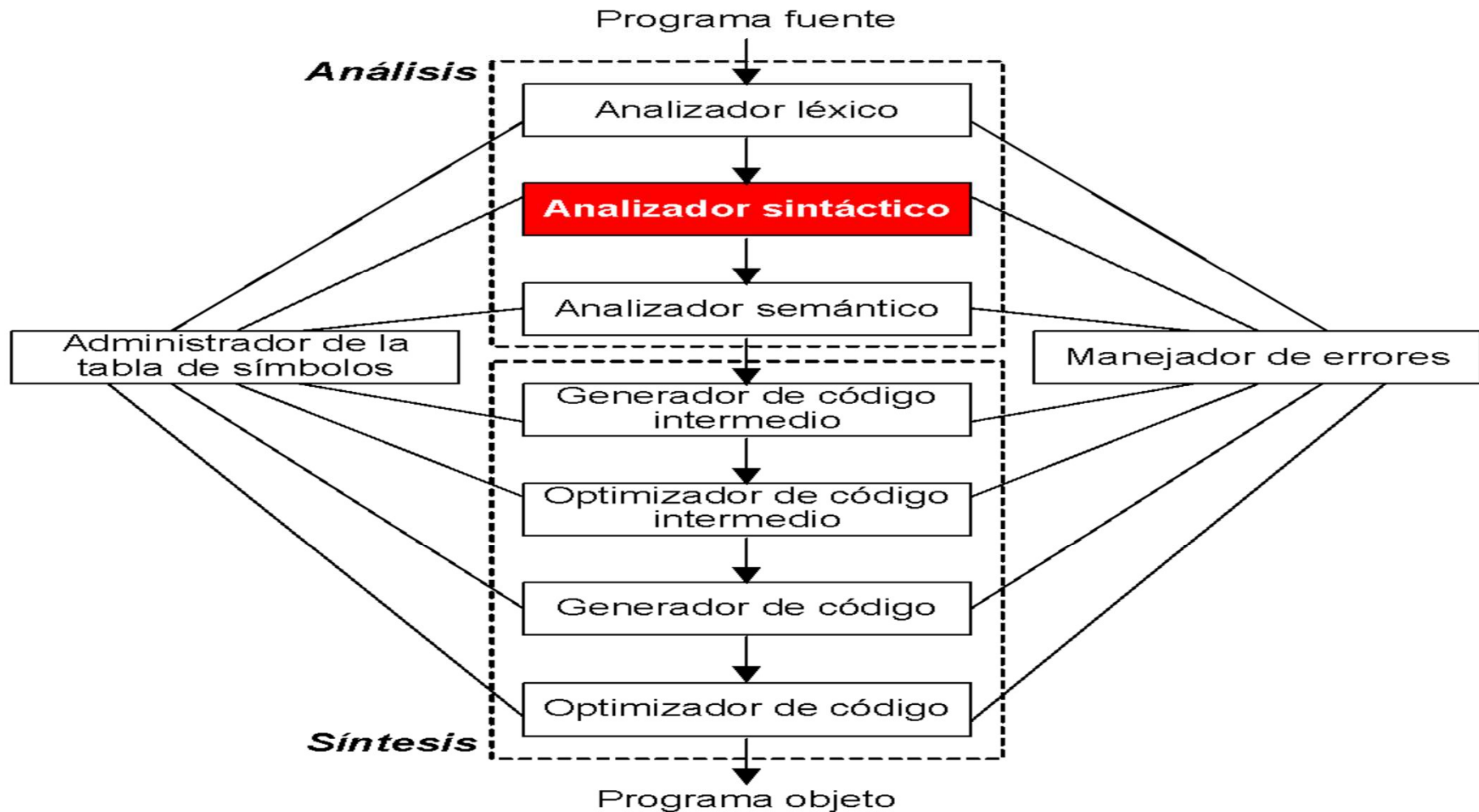
- Orden de aplicación de los Algoritmos de limpieza y bien formar GIC
 1. Eliminación de Reglas innecesarias
 2. Eliminación de Reglas no generativas
 3. Eliminar Reglas de Redenominación (Menos la regla del axioma $S \rightarrow \lambda$)
 4. Eliminación de S. Superfluos
 5. Eliminación de S. Inaccesibles

Utilización de las gramáticas en un compilador

(caso de uso)

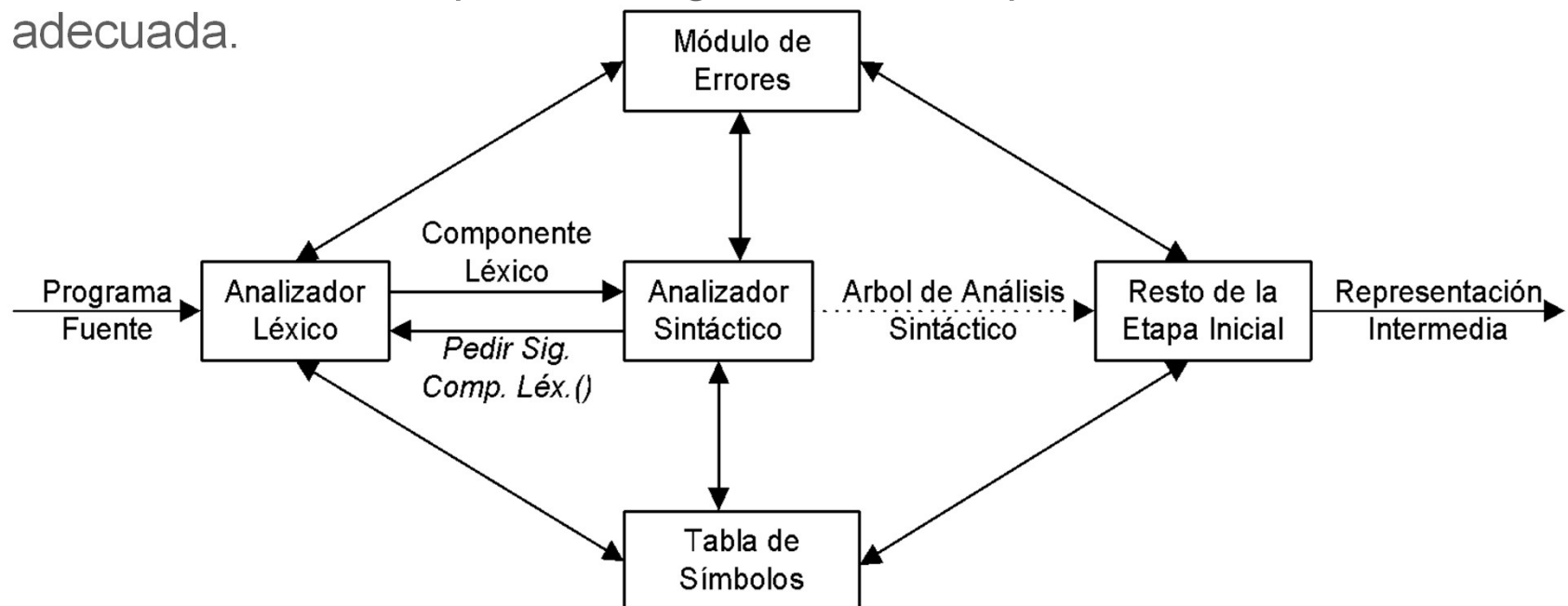


Introducción



Introducción

- Papel del Analizador sintáctico.
 - Obtiene una cadena de componentes léxicos del analizador léxico y comprueba si dicha cadena puede ser generada por la gramática del lenguaje fuente y “genera el árbol de análisis sintáctico”.
 - Debe informar si se produce algún error, recuperándose de forma adecuada.

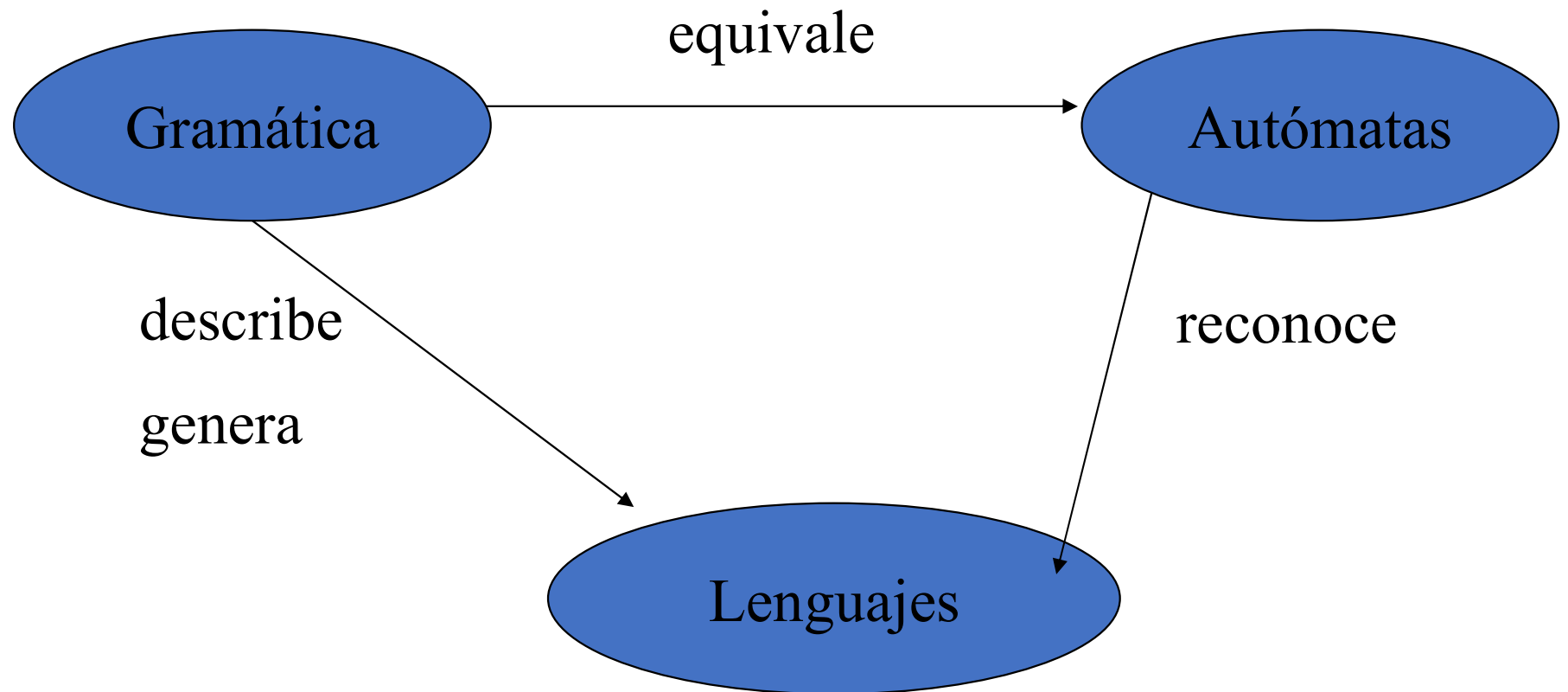


Analizador Sintáctico. Tipos

- Tres tipos generales de analizadores sintácticos:
 - **Métodos Universales:** Cocke-Younger-Kasami (CYK) y Earley
 - Sirven para cualquier gramática
 - Muy ineficientes
 - **Descendentes** (*top-down*)
 - Construyen el árbol de análisis sintáctico desde arriba (raíz, axioma) hasta abajo (hojas, terminales)
 - Analizadores Descendentes Recursivos
 - Analizadores LL(1) con tabla
 - **Ascendentes** (*bottom-up*)
 - Construyen el árbol de análisis sintáctico desde abajo hacia arriba
 - Analizadores de Precedencia de Operador
 - Analizadores LR(1)



Autómatas - Gramáticas Formales



3.1. Gramáticas Formales

- Definición: **Concepto formal de gramática.** Se llama gramática formal a la cuádrupla

$$G = (\Sigma_T, \Sigma_N, S, P)$$

donde:

- Σ_T , alfabeto de símbolos terminales
- Σ_N , alfabeto de símbolos no terminales
 - Notación: $\Sigma = \Sigma_T \cup \Sigma_N$
- $S \in \Sigma_N$, es el axioma, símbolo inicial o símbolo distinguido
- P es un conjunto finito de reglas de producción de la forma $u \rightarrow v$, donde $u \in \Sigma^+$ y $v \in \Sigma^*$
- Se verifica además que $\Sigma_T \cap \Sigma_N = \varnothing$ y el alfabeto sobre el que trabajaremos será $\Sigma = \Sigma_T \cup \Sigma_N$



3.1. Gramáticas Formales

- Definición: **Derivación**

- Dada una gramática $G=(\Sigma_T, \Sigma_N, S, P)$ y dos palabras $a, b \in \Sigma^*$, decimos que b es **derivable a partir de a en un paso** ($a \vdash b$) si y solo si existen palabras $d_1, d_2 \in \Sigma^*$ y una producción $j \rightarrow g$ tales que $b = d_1 g d_2$ y $a = d_1 j d_2$
- Dada una gramática $G=(V, T, P, S)$ y dos palabras $a, b \in \Sigma^*$, decimos que b es **derivable a partir de a** ($a \vdash^* b$) si y solo si existen una sucesión de palabras $d_1, d_2, \dots, d_n \in \Sigma^*$ tales que $a = d_1 \vdash d_2 \vdash \dots \vdash d_n = b$.

- En este caso se dice que existen una **relación de Thue entre a y b**



3.1 Gramáticas Formales.

Derivaciones

- **Derivación Más a la Izquierda.** Cuando se utiliza en cada derivación directa la producción aplicada a los símbolos más a la izquierda de la palabra.
- **Derivación Más a la Derecha.** Cuando se utiliza en cada derivación directa la producción aplicada a los símbolos más a la derecha de la palabra.



3.1. Gramáticas Formales

Definición: **Formas sentenciales y sentencias**

- Sea una gramática $G = (\Sigma_T, \Sigma_N, S, P)$. Una palabra $x \in \Sigma^*$ se denomina **forma sentencial** de G si se verifica que

$$S \vdash^* x$$

es decir, existe una **relación de Thue** entre el axioma de la gramática y la palabra.

- Si una forma sentencial x cumple que $x \in \Sigma_T^*$, se dice que x es una **sentencia** o instrucción de G . Es decir, las sentencias estarán compuestas únicamente por símbolos terminales.



3.1. Gramáticas Formales

- Definición: Sea G una gramática y $v = xuy$ una de sus formas sentenciales. Se dice que u es una **frase** de la forma sentencial v , respecto del símbolo no terminal U , si

$$\begin{aligned} S & \mid\!\!\!-\!^* xUy \\ U & \mid\!\!\!-\!^+ u \end{aligned}$$



3.1. Gramáticas Formales

- Definición 3.5: Se dice que u es una **frase simple** de la forma sentencial $v = xuy$, respecto del símbolo no terminal U , si

$$\begin{array}{l} S \mid\!\!\!\vdash^* xUy \\ U \rightarrow u \end{array}$$

- Notación simplificada

Si existen dos reglas de la forma:

$$U \rightarrow V$$

$$U \rightarrow W$$

Se pueden representar de la forma :

$$U \rightarrow V \mid W$$



3.1. Gramáticas Formales

- Ejemplo: Consideremos la gramática definida por:

$$\Sigma_T = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0\}$$

$$\Sigma_N = \{N, C\}$$

$$S = N$$

$$P = \{ N ::= NC, N ::= C, C ::= 0, C ::= 1, C ::= 2, C ::= 3, \\ C ::= 4, C ::= 5, C ::= 6, C ::= 7, C ::= 8, C ::= 9 \}$$

P se puede también se puede representar como:

$$P = \{ \begin{array}{l} N \rightarrow NC \mid C \\ C \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \end{array} \}$$



3.1. Gramáticas Formales

- Árboles de derivación: Algunas gramáticas se pueden representar mediante un árbol de derivación. Este árbol se construye así:
 - a) La raíz del árbol corresponde al axioma de la gramática
 - b) Una derivación directa se representa por un conjunto de ramas que salen de un nodo determinado. Al aplicar una regla, uno de los símbolos de la parte izquierda de la producción deriva en la palabra de la parte derecha. Se dibuja una rama por cada símbolo de la parte dcha., que parte del nodo correspondiente al símbolo de la izda.



3.1. Gramáticas Formales

- En cada rama, el nodo de partida se denomina *padre*. El final se denomina *hijo* del primero
- A lo largo del proceso de construcción del árbol, los nodos finales de cada paso, leídos de izquierda a derecha, forman **la forma sentencial** obtenida por la derivación representada por el árbol.
- Será rama *terminal* aquella dirigida hacia un símbolo terminal. Este nodo se llama *hoja*. El conjunto de *hojas*, leído de izquierda a derecha, forma la sentencia generada por la derivación.



3.1. Gramáticas Formales

- Ejemplo: Sea la gramática

$G = (\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\},$

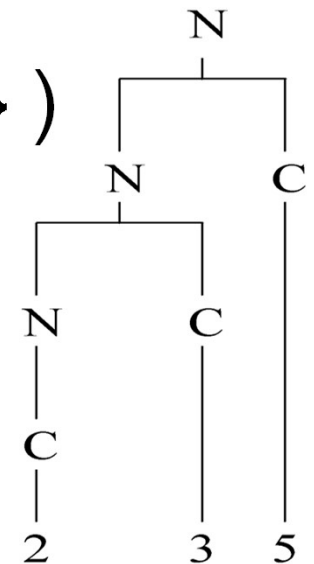
$\{N, C\},$

$N,$

$\{N \rightarrow NC \mid C,$

$C \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9\})$

Árbol de derivación de 235:



3.1. Gramáticas Formales

- Subárbol: Dado un árbol correspondiente a una derivación, se denomina **subárbol** al árbol cuya raíz es un nodo cualquiera y cuyos nodos son los descendientes de éste.
- Teorema Los nodos terminales de un subárbol, leídos de izquierda a derecha, forman una frase respecto de la raíz del subárbol.



3.1. Gramáticas Formales

- Proceso Inverso: Para cada derivación existe un único árbol de derivación. Sin embargo, de una misma sentencia pueden obtenerse, a veces, varias derivaciones diferentes.

Por ejemplo, el mismo árbol anterior puede aplicarse a las siguientes derivaciones:

$N \mid\text{---} NC \mid\text{---} NCC \mid\text{---} CCC \mid\text{---} 2CC \mid\text{---} 23C \mid\text{---} 235$

$N \mid\text{---} NC \mid\text{---} NCC \mid\text{---} CCC \mid\text{---} C3C \mid\text{---} 23C \mid\text{---} 235$

$N \mid\text{---} NC \mid\text{---} NCC \mid\text{---} CCC \mid\text{---} 2CC \mid\text{---} 2C5 \mid\text{---} 235$



3.1. Gramáticas Formales

- Definición:

- Ambigüedad: Si para una misma sentencia podemos tener varios árboles de derivación diferentes, se dice que la sentencia es **ambigua**.

- Ejemplo:

$$G = (\{i, +, *, (,)\}, \{E\}, E, E ::= E + E \mid E * E \mid (E) \mid i \})$$

- Consideremos la sentencia : $i+i*i$.

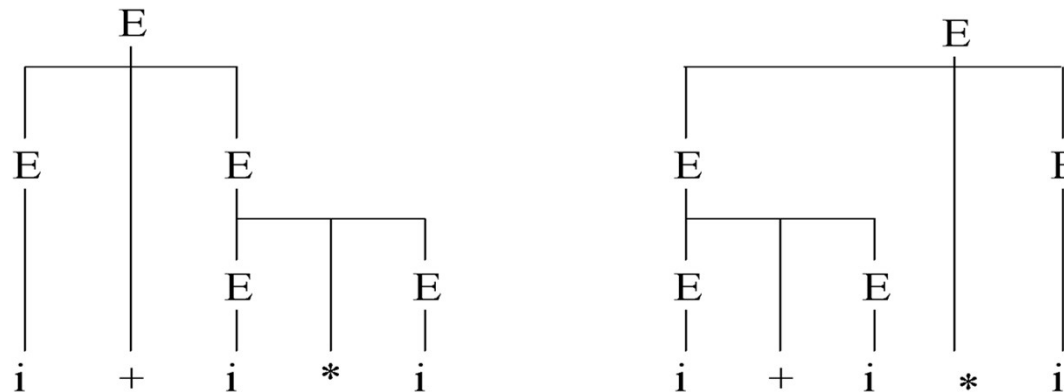


3.1. Gramáticas Formales

- Ejemplo:

$G = (\{i, +, *, (,)\}, \{E\}, E, E ::= E + E \mid E * E \mid (E) \mid i)$

- Consideremos la sentencia : **$i+i*i$**



3.1. Gramáticas Formales

- Definiciones:
 - Gramática ambigua: si tiene dos o más árboles de derivación distintos para la misma cadena.
 - Lenguaje ambiguo: Si existe al menos una gramática ambigua que lo genere.
 - Lenguaje inherentemente ambiguo: si todas las gramáticas que lo generan son ambiguas.



3.1. Gramáticas Formales

- Definición:

- Lenguaje generado por una gramática G:

Se representa por $L(G)$ y es el conjunto de todas las sentencias de la gramática; es decir, todas las palabras se pueden obtener desde el axioma por aplicación de derivaciones.

$$L(G) = \{x \mid S \xrightarrow{*} x, x \in \Sigma_T^*\}$$



3.1. Gramáticas Formales

- Definición:
 - Equivalencia de gramáticas: Dadas dos gramáticas G_1 y G_2 decimos que son equivalentes, si $L(G_1) = L(G_2)$, es decir, si generan el mismo lenguaje.

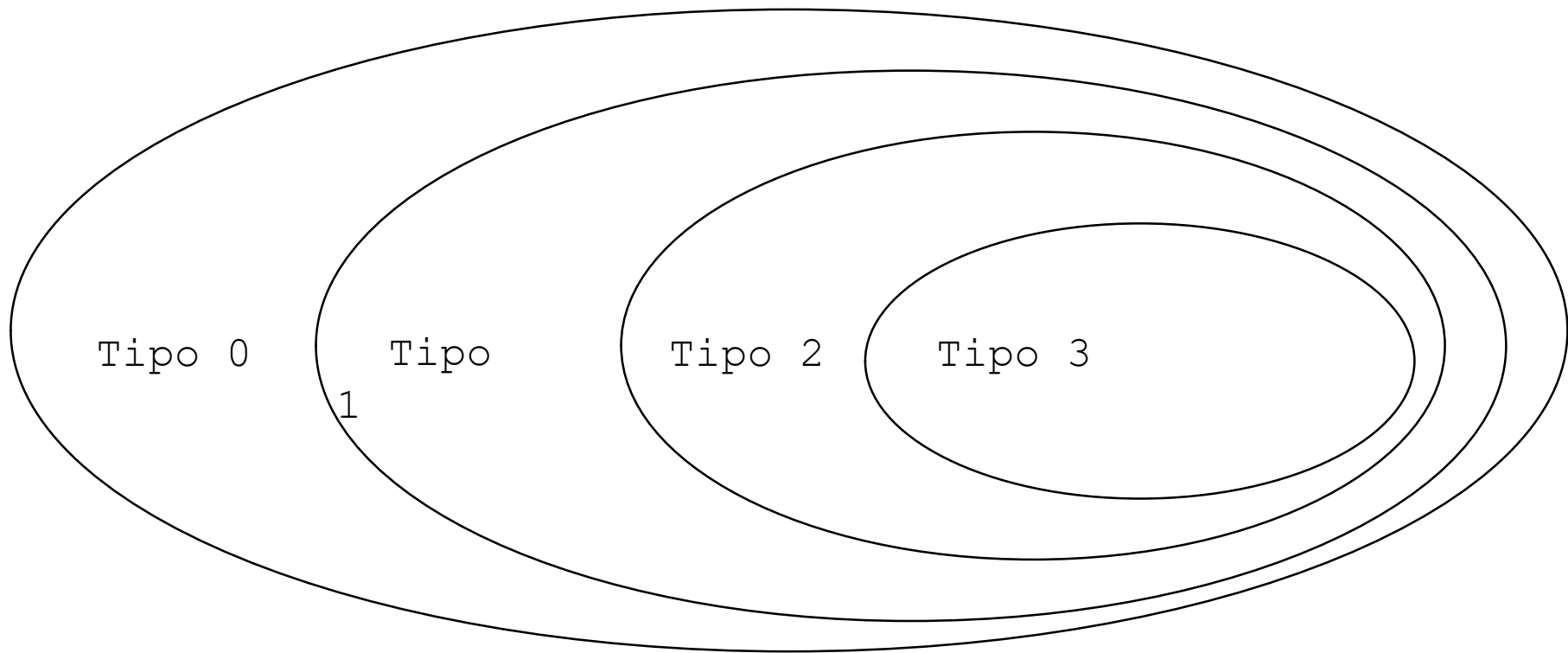


3.1. Gramáticas Formales

- Definición:
 - Regla compresora: Es aquella cuya parte derecha está formada por menos elementos que la parte izquierda
 - Ejemplos:
 - $0C0 \rightarrow 1$
 - Palabra vacía a la derecha:
 $A \rightarrow \lambda$



Jerarquía de Chomsky



Jerarquía de Chomsky

Gramáticas	Lenguajes	Máquinas
Sin restricciones o de Tipo 0	Sin restricciones o de Tipo 0	Máquina de Turing
Sensible al contexto o de Tipo 1	Sensible al contexto o de Tipo 1	Autómata linealmente acotado
Independiente del contexto o de Tipo 2	Independiente de contexto o de Tipo 2	Autómata de pila
Regular o de Tipo 3	Regular o de Tipo 3	Autómata Finito



Gramáticas de Tipo 0

(Con estructura de frase o sin restricciones)

Sus producciones son de la forma:

$$\alpha \rightarrow \beta,$$

donde

$$\alpha \in (\Sigma_N \cup \Sigma_T)^* \Sigma_N (\Sigma_N \cup \Sigma_T)^*$$

y

$$\beta \in (\Sigma_N \cup \Sigma_T)^*$$



Gramáticas de Tipo 1

(Dependientes o sensibles del contexto)

- Sus producciones son de la forma:

$$\alpha_1 A \alpha_2 \rightarrow \alpha_1 \beta \alpha_2$$

donde

$$\alpha_1, \alpha_2 \in (\Sigma_N \cup \Sigma_T)^*$$

$$A \in \Sigma_N$$

$$\beta \in (\Sigma_N \cup \Sigma_T)^+$$

excepto la regla $S \rightarrow \lambda$ en cuyo caso S no aparece en la parte derecha de ninguna regla.



Gramáticas de Tipo 2

(Gramáticas Libres de Contexto)

- Sus producciones son de la forma:

$$A \rightarrow \alpha$$

donde

$$A \in \Sigma_N$$

$$\alpha \in (\Sigma_N \cup \Sigma_T)^+$$

o bien es la regla $S \rightarrow \lambda$

Nota: se suele relajar y permitir $\alpha \in (\Sigma_N \cup \Sigma_T)^*$



Gramáticas de Tipo 3

(Gramáticas regulares o de estado finito)

- Sus producciones son de la forma:

$$A \rightarrow uB \text{ o } A \rightarrow Bu \text{ o } A \rightarrow u$$

donde

$$A, B \in \Sigma_N \text{ y } u \in \Sigma_T$$

Se suelen permitir producciones de la forma $S \rightarrow \lambda$





- El **método de las derivadas** va a permitir resolver el problema de síntesis de Kleene (se verá más adelante), generando el autómata que acepta el lenguaje descrito por cualquier expresión regular ($ER \rightarrow AF$) en tres pasos:
 - 1) Calcular derivadas de la ER
 - 2) Calcular gramática de tipo 3 que genera el lenguaje representado por la ER ($ER \rightarrow G$)
 - 3) Transformar la gramática al AF ($GR \rightarrow AF$)



Cálculo de derivadas

- Definición 3.11: Derivada de una ER α respecto de un símbolo $a \in \Sigma$:

$$D_a(\alpha) = \{x \mid ax \in \alpha\}$$

- Cálculo:

- $D_a(\emptyset) = \emptyset$

- $D_a(\lambda) = \emptyset$

- $D_a(a) = \lambda$

- $D_a(b) = \emptyset, \forall b \in \Sigma, b \neq a$

- $D_a(\alpha + \beta) = D_a(\alpha) + D_a(\beta)$

- $D_a(\alpha.\beta) = D_a(\alpha).\beta + \delta(\alpha).D_a(\beta)$ donde

$$\delta(\alpha) = \begin{cases} \lambda, & \text{si } \lambda \in \alpha \\ \emptyset, & \text{si } \lambda \notin \alpha \end{cases}$$

- $D_a(\alpha^*) = D_a(\alpha).\alpha^*$



Cálculo de derivadas

- Composición de derivadas:

- $D_{ab}(\alpha) = D_b(D_a(\alpha))$

- 1) Calcular derivadas de la ER= α_0 :

- a) $\forall a \in \Sigma$:

- Calcular de $\alpha_i = D_a(\alpha_0)$, empezando por $i=1$.

- b) Para cada α_i nuevo generado:

- Calcular $\forall a \in \Sigma$, calcular $\alpha_j = D_a(\alpha_i)$

- c) Hasta que no se generen nuevos α_i



Creación de una gramática

2) Construiremos una gramática G a partir de una ER ($ER \rightarrow G$)

$$G = (\Sigma, \alpha_0 \cup \alpha_i, \alpha_0, P)$$

Donde P se construye así:

a) Si $D_a(\alpha) = \beta$, $\beta \neq \lambda$, $\beta \neq \emptyset$

Crear regla: $\alpha \rightarrow a\beta$

b) Si $\lambda \in D_a(\alpha)$

Crear regla: $\alpha \rightarrow a$

c) Si $\lambda \in \alpha_0$

Crear regla: $\alpha_0 \rightarrow \lambda$

d) Si $D_a(\alpha) = \emptyset$

No generar ninguna regla



Obtención del AF

2) Obtención del AF. Dos opciones:

2.1) Desde las derivadas: $ER \rightarrow AF$

$$AF = (\Sigma, \alpha_i \cup \{q_f\}, \delta, \alpha_0, \{q_f\})$$

Donde δ se construye así:

a) Si $D_a(\alpha) = \beta$, $\beta \neq \lambda$, $\beta \neq \emptyset$

Crear transición: $\delta(\alpha, a) = \beta$

b) Si $\lambda \in D_a(\alpha)$

$$q_f \in \delta(\alpha, a)$$

c) Si $\lambda \in \alpha_0$

$$q_f \in \delta(\alpha_0, \lambda)$$

d) Si $D_a(\alpha) = \emptyset$

$$\delta(\alpha, a) = \emptyset$$



Obtención del AF

2.2) Desde la gramática: $GR \rightarrow AF$

Sea $G = (\Sigma_T, \Sigma_N, S, P) \rightarrow AF = (\Sigma_T, \Sigma_N \cup \{q_f\}, \delta, S, \{q_f\})$

Donde δ se construye así:

a) Para cada $p \in P$: $\alpha \rightarrow a\beta$ / $\alpha, \beta \in \Sigma_N, a \in \Sigma_T$

$$\delta(\alpha, a) = \beta$$

b) Para cada $p \in P$: $\alpha \rightarrow a$ / $\alpha \in \Sigma_N, a \in \Sigma_T$

$$q_f \in \delta(\alpha, a)$$

c) Si $\{S \rightarrow \lambda\} \in P$

$$\delta(S, \lambda) = q_f$$



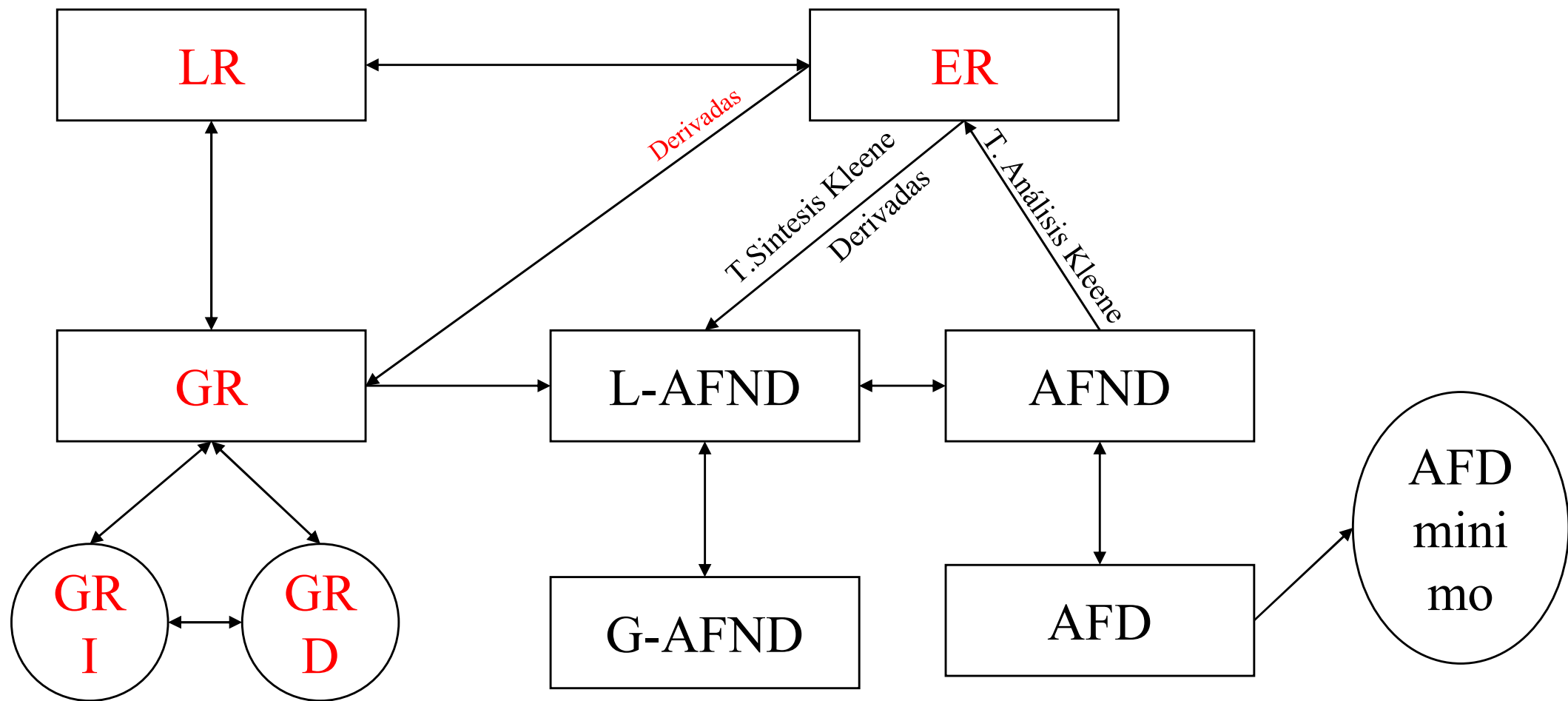
Estado actual de la asignatura

Sea L un lenguaje. Son equivalentes las siguientes afirmaciones:

- i) L es LR
- ii) $\exists \alpha \in ER / L(\alpha)=L$
- iii) $\exists M \in AF / L(M)=L$ (spoiler)
- iv) $\exists G \in GR / L(G)=L$



Esquema general de la asignatura



Autómatas

Lenguajes Independientes del Contexto (LIC)

(Simplificación de gramáticas)



Árboles de Derivación

- Derivaciones.
 - Una derivación es la secuencia de producciones que hay que aplicar, partiendo del símbolo inicial, para generar una cadena del lenguaje.
 - Una cadena pertenece a un lenguaje si se puede construir una derivación que la genere.
 - Paso simple de derivación: Si $a \rightarrow \gamma$, es una producción y α y β son cadenas arbitrarias de símbolos gramaticales, terminales y no terminales:
 $\alpha a \beta \rightarrow \alpha \gamma \beta$.



Árboles de Derivación

- Ejemplo:

- Gramática:

$$E \rightarrow E O E \mid (E) \mid - E \mid \text{num}$$
$$O \rightarrow + \mid * \mid -$$

- Entrada:

$$34 * (26 + 12)$$

- Cadena de tokens:

$$\text{num} * (\text{num} + \text{num})$$


Árboles de Derivación

- Secuencia de derivaciones.

$$\begin{array}{lll} & & E \\ E \rightarrow E \ O \ E & \Rightarrow & E \ O \ E \\ E \rightarrow \text{num} & \Rightarrow & \text{num} \ O \ E \\ O \rightarrow * & \Rightarrow & \text{num} * E \\ E \rightarrow (\ E \) & \Rightarrow & \text{num} * (\ E \) \\ E \rightarrow E \ O \ E & \Rightarrow & \text{num} * (\ E \ O \ E \) \\ E \rightarrow \text{num} & \Rightarrow & \text{num} * (\ \text{num} \ O \ E \) \\ O \rightarrow + & \Rightarrow & \text{num} * (\ \text{num} + E \) \\ E \rightarrow \text{num} & \Rightarrow & \text{num} * (\ \text{num} + \text{num} \) \end{array}$$



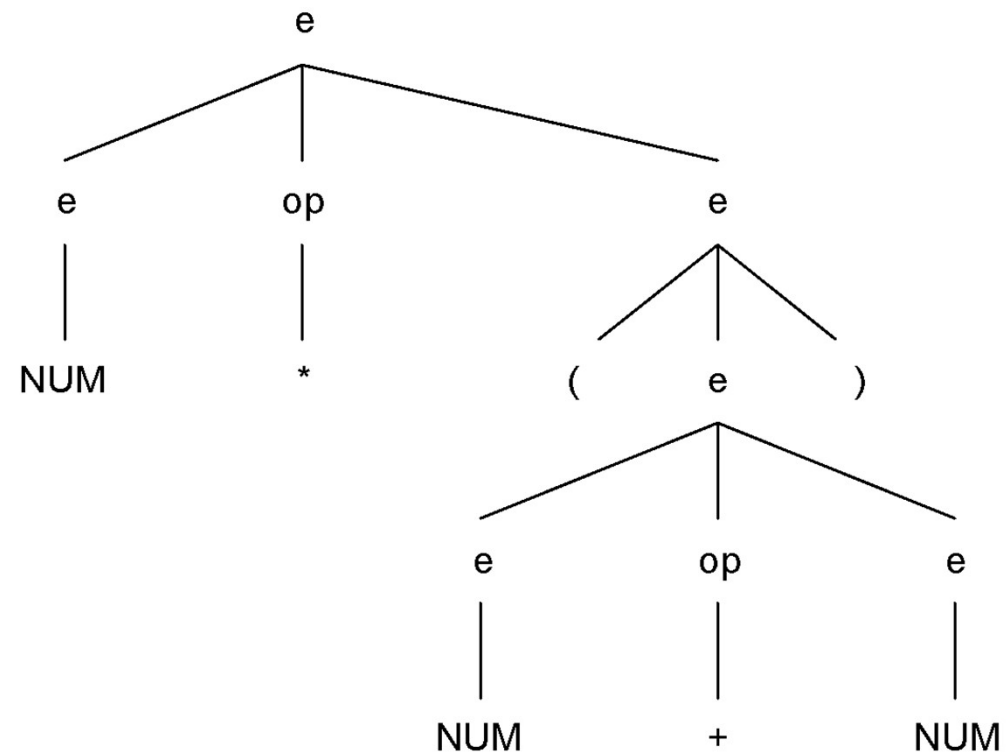
- **Árbol de Análisis Sintáctico.**

- Es una representación gráfica de la estructura sintáctica.
- Muestra la secuencia de derivaciones realizada, pero no el orden.
 - Los nodos interiores se corresponden con no terminales.
 - Las hojas son terminales.
 - Cada nodo padre es la parte izquierda de una producción, y sus hijos son la parte derecha, de izq a dcha.



Árboles de Derivación

- Ejemplo: $\text{num} * (\text{num} + \text{num})$



Árboles de Derivación

- **¿Qué es una derivación?**

Se representa por $x \Rightarrow y$ a la aplicación de una regla de producción $a \rightarrow b$ a una cadena x para convertirla en otra cadena o palabra y .

Cuando lo que se aplica es una **secuencia de producciones** a una cadena se representa por $x \Rightarrow^* y$, queriendo indicar que llegamos a otra cadena en más de un paso. En el ejemplo anterior, $D \rightarrow T V \Rightarrow^* \text{int } a, b, c;$, y realizamos, por tanto, derivaciones en más de un paso para obtener la cadena final

- **Derivación más a la izquierda frente a derivación más a la derecha.**

- Derivación más a la izquierda.
Derivar el no terminal más a la izquierda.
El ejemplo anterior utilizaba derivación más a la izquierda.
- Derivación más a la derecha.
Derivar el no terminal más a la derecha.



Árboles de Derivación

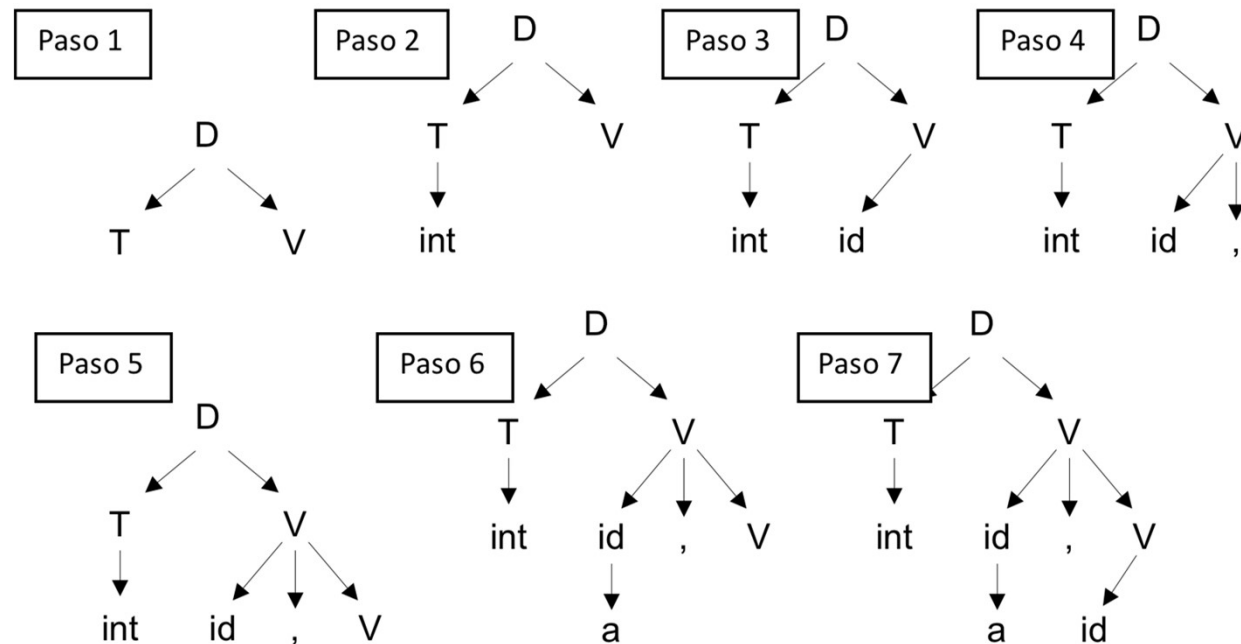
Supongamos la siguiente sentencia para la gramática de declaración de variables de la gramática que ya hemos utilizado anteriormente: **int a, b;**

$D \rightarrow T V$

$T \rightarrow \text{int} \mid \text{real} \mid \text{char}$

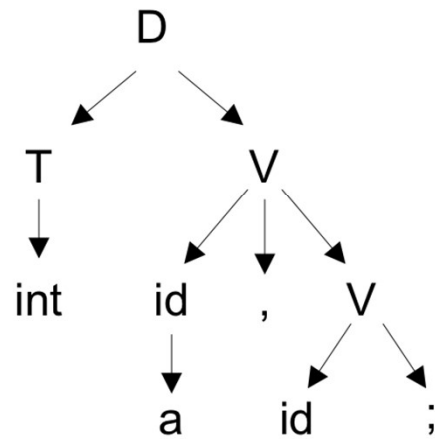
$V \rightarrow \text{id}; \mid \text{id}, V$

Para validar esta sentencia vamos a hacer derivaciones por la izquierda

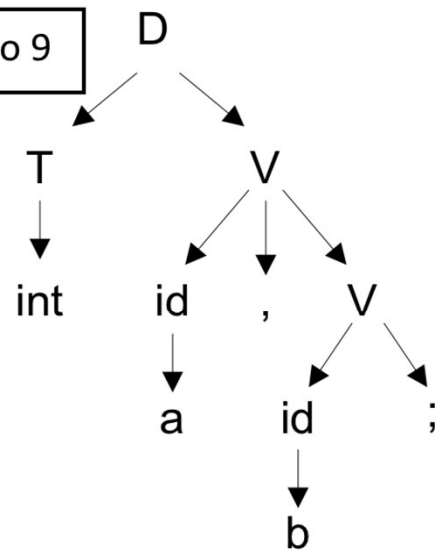


Árboles de Derivación

Paso 8



Paso 9



Limpieza de Gramáticas

- Las gramáticas que nos permiten definir lenguajes de programación están formadas por una gran cantidad de reglas en notación BNF que puede contener errores, bien porque no son necesarias, bien porque tienen símbolos superfluos o innecesarios. Al proceso de corregir la gramática se le denomina “**limpieza de la gramática**”. El objetivo es que tanto los símbolos como las reglas que se utilizan en la gramática sirvan para algo.
- Hay dos tipos de errores que se cometen con los símbolos: símbolos superfluos y símbolos inaccesibles.
- **Símbolos superfluos:** Depende del conjunto al que pertenezcan (terminales o no terminales)
 - **Símbolo no terminal superfluo:** Es aquel del que se derivan sentencias que no contienen ningún terminal
 - **Símbolo terminal superfluo:** Es aquel que no puede ser alcanzado por derivación desde el axioma
- **Símbolos inaccesibles:** Son aquellos símbolos no terminales que no pueden ser alcanzados por derivaciones desde el axioma de la gramática.



Limpieza de Gramáticas

Hay tres tipos de errores que se cometen con la reglas y son aquellas que no generan derivaciones útiles: **reglas innecesarias**, **reglas no generativas** y **reglas de red denominación**

- **Reglas innecesarias:** Son reglas del tipo $A \rightarrow A$, donde A pertenece a los no terminales
- **Reglas no generativas:** Son reglas del tipo $A \rightarrow \lambda$, cuando A es distinta de S (el axioma de la gramática)
- **Reglas de red denominación:** Son reglas del tipo $A \rightarrow B$, donde tanto A como B pertenecen a los no terminales.



Limpieza de Gramáticas

Una gramática **está bien formada si está limpia**, es decir:

- Sin reglas innecesarias ($A \rightarrow A$)
- Sin símbolos inaccesibles
- Sin símbolos superfluos
- Sin reglas no generativas ($A \rightarrow \lambda$)
- Sin reglas de red denominación ($A \rightarrow B$)



Limpieza de Gramáticas

Ejemplo: $G: (\{a, b, c, d\}, \{X, Y, V, W, Z\}, X, P)$, donde P está compuesto por las siguientes reglas o producciones

$X \rightarrow Wa \mid Zba \mid \lambda$

$Y \rightarrow aVa$

$V \rightarrow Y$

$W \rightarrow bX$

$Z \rightarrow Za$

Observando la gramática se puede determinar:

- No hay reglas innecesarias.
- Los símbolos Y y V son inaccesibles, puesto que no podemos llegar a ellos mediante derivaciones desde el axioma de la gramática (X).
- El símbolo Z es un símbolo no terminal superfluo $Z \rightarrow Za$ puesto que cuando se hagan derivaciones siempre aparece el propio símbolo Z .
- Los símbolos c y d son símbolos terminales superfluos.
- No hay reglas no generativas
- La regla $V \rightarrow Y$ es una regla de red denominación (este error es muy común)
- Para limpiar esta gramática eliminamos tanto los símbolos inaccesibles y superfluos como las producciones en las que estos aparecen. Lo mismo se hace con las reglas innecesarias y con ambas eliminaciones no se altera el lenguaje generado por la gramática.



Limpieza de Gramáticas

Si limpiamos esta gramática nos queda $G: (\{a, b, \}, \{X, W\}, X, P)$

$$X \rightarrow Wa \mid \lambda$$

$$Y \rightarrow aVa$$

$$V \rightarrow V$$

$$W \rightarrow bX$$

$$Z \rightarrow Za$$

Obteniéndose finalmente:

$$X \rightarrow Wa \mid \lambda$$

$$W \rightarrow bX$$

NOTA: El objetivo, no es tanto el dominar los algoritmos de limpieza de gramáticas, sino el reconocer los errores y no cometerlos cuando diseñemos una gramática



Gramáticas

- Ejemplo: Construir una GIC que reconozca secuencias de proposiciones separadas por símbolos de punto y coma en los bloques *begin-end* de Pascal.
 - Debe existir al menos una proposición.

bloque --> BEGIN lista_props END

lista_props --> prop ; lista_props | prop

- Puede existir una secuencia de proposiciones vacía.

bloque --> BEGIN lista_props END

lista_props_opc --> lista_props | λ

lista_props --> prop ; lista_props | prop



Ambigüedad

La gramática anterior es ambigua.

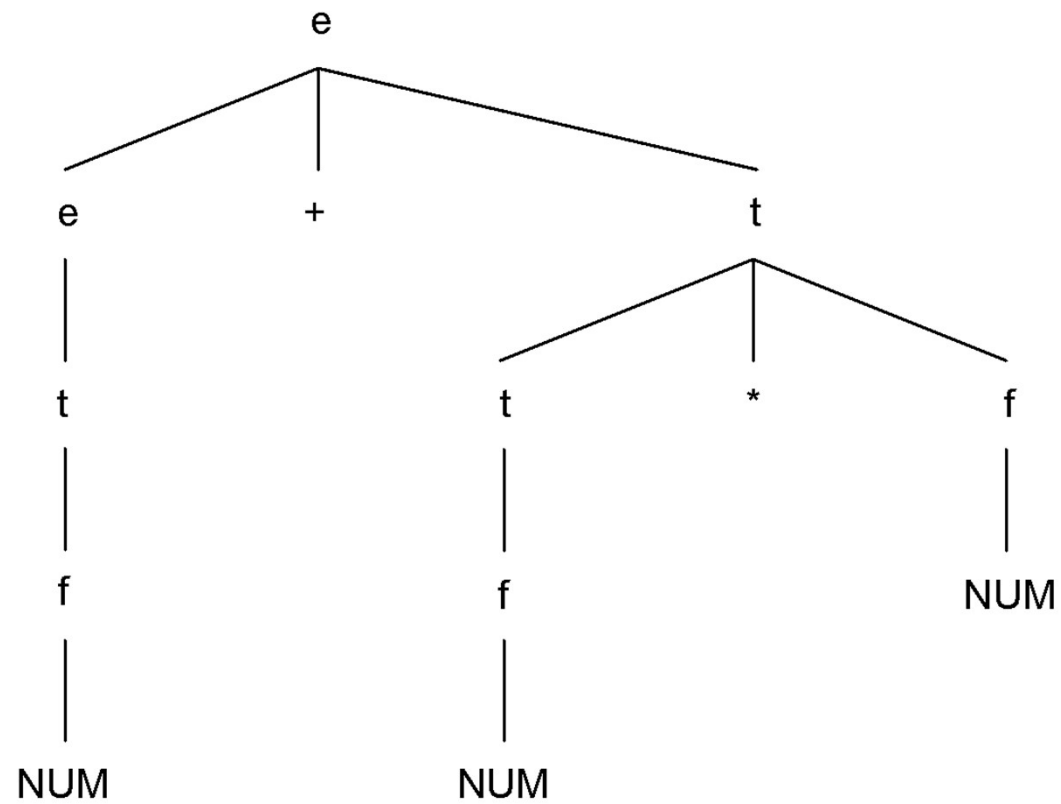
- La aplicación de diferentes órdenes de derivación produce diferentes árboles sintácticos.
 - Los resultados obtenidos con cada orden de derivación son diferentes → Sólo uno de ellos puede ser el correcto.
 - En ocasiones se puede eliminar la ambigüedad rescribiendo la gramática con nuevos símbolos no terminales.
- Ejemplo: Modificar la gramática de las expresiones aritméticas para que no exista ambigüedad.

Prioridad de operadores.

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow \text{num} \mid (E)$$


Ambigüedad

Árbol de análisis sintáctico para la cadena: num + num * num



Ambigüedad

- Ejemplo: Razonar si la siguiente gramática es ambigua.

prop --> IF exp THEN prop

prop --> IF exp THEN prop ELSE prop

prop --> IDENTIFICADOR := NUMERO ;

exp --> IDENTIFICADOR = IDENTIFICADOR

- ¿Existe un único árbol para la siguiente cadena?

IF a=b THEN

IF a=c THEN

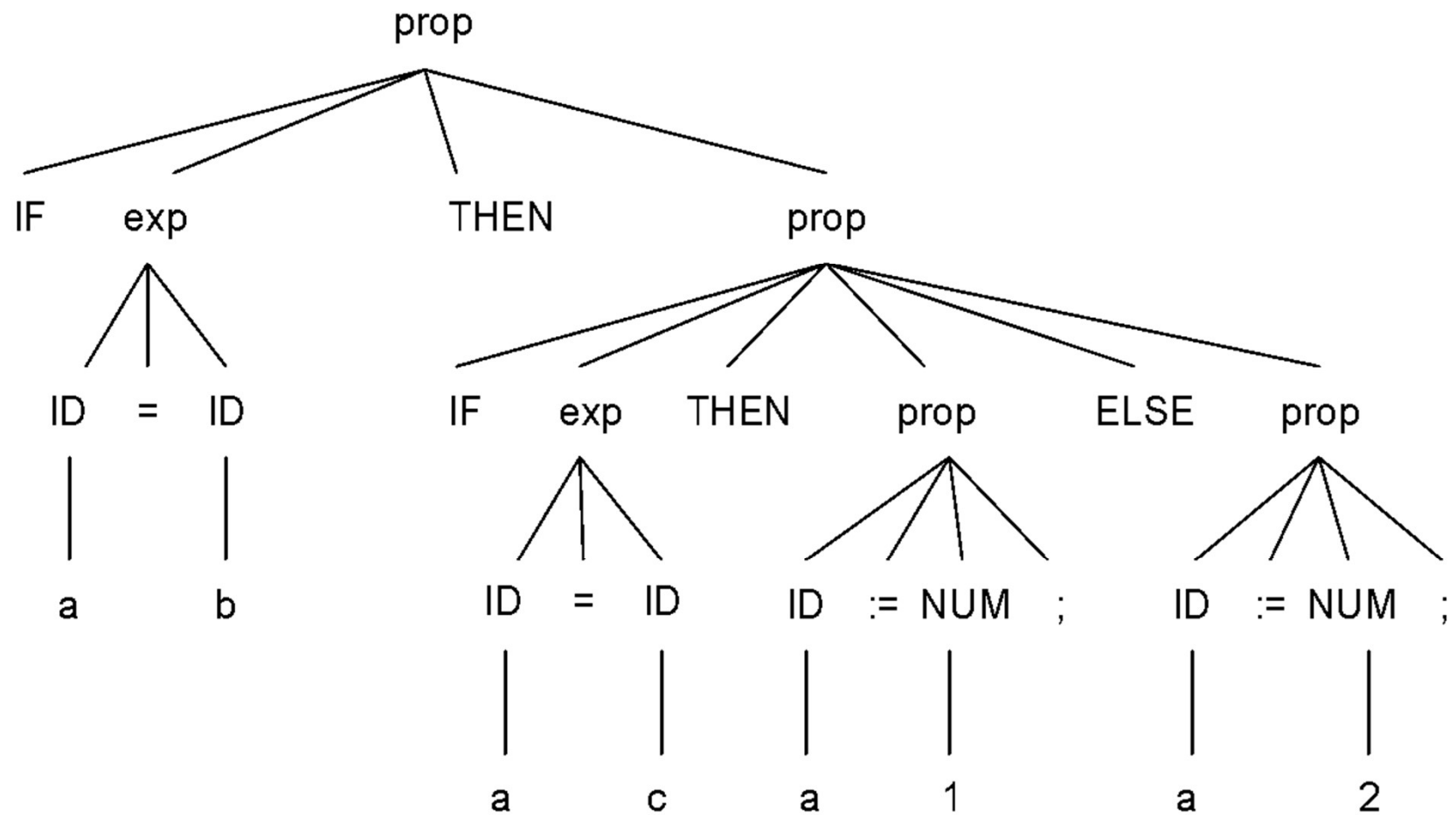
a:=1;

ELSE

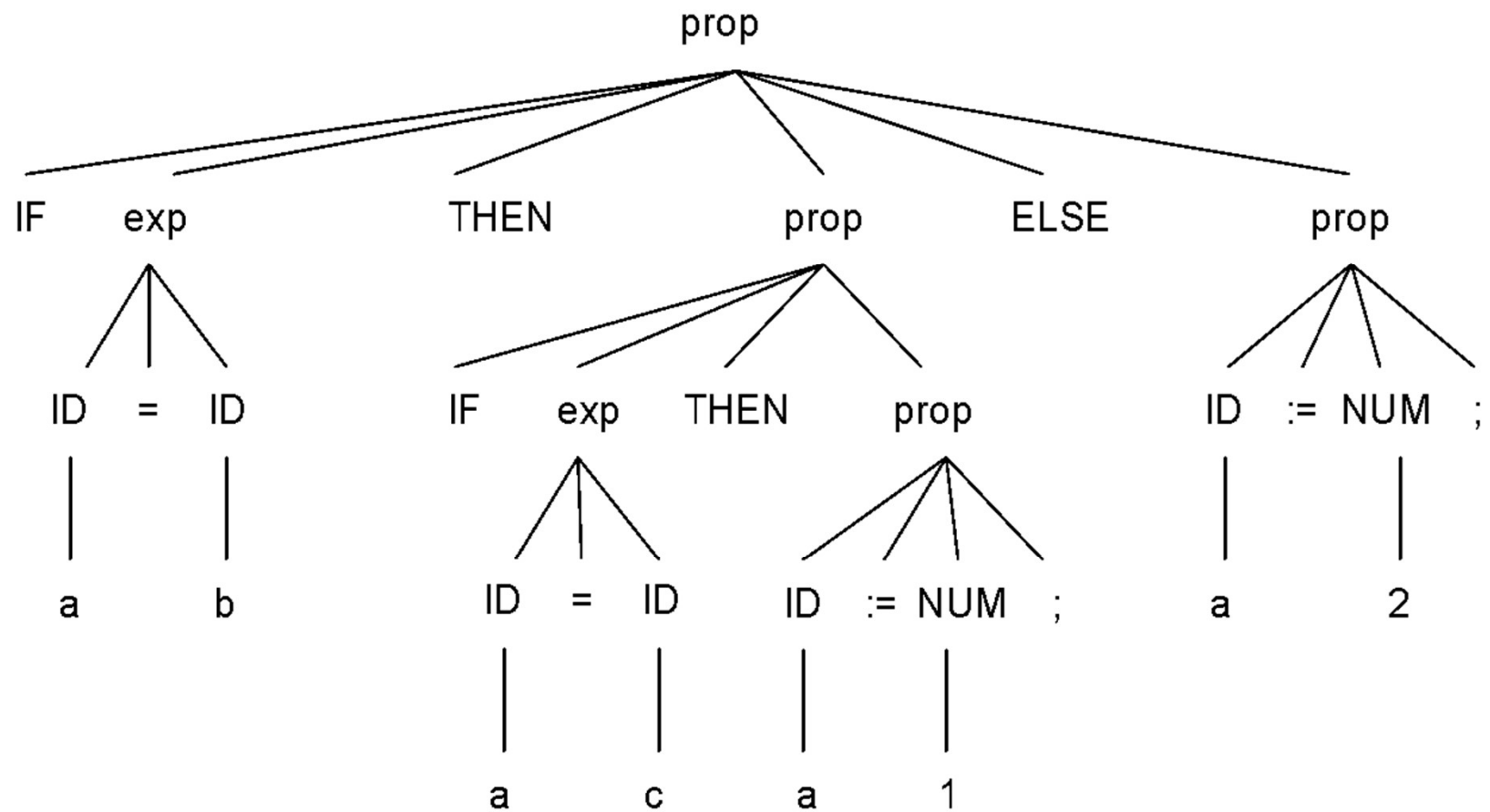
a:=2;



Ambigüedad



Ambigüedad



Ambigüedad

- ¿Cómo podríamos modificar la anterior gramática para que no sea ambigua?

prop --> prop2

prop --> prop3

prop2 --> IF exp THEN prop

prop3 --> IDENTIFICADOR := NUMERO ;

prop3 --> IF exp THEN prop3 ELSE prop



Ambigüedad. Recursividad

- Una gramática recursiva por la izquierda puede hacer que un AS por descenso recursivo, con o sin retroceso, entre en un bucle infinito.
 - Cuando se intenta expandir a , puede que de nuevo se intente expandir a sin haber consumido ningún símbolo de la entrada.
 - Ejemplo:

$$\begin{aligned}s &\rightarrow rA \mid B \\ r &\rightarrow rC \mid D\end{aligned}$$

- Debemos eliminar la recursividad directa por la izquierda. Para ello, reescribiremos la producción de otra forma.

$$r \rightarrow r\alpha \mid \beta$$

se sustituye por:

$$r \rightarrow \beta r'$$

$$r' \rightarrow \alpha r' \mid \lambda$$



Ambigüedad. Recursividad

- Ejemplo: Gramática para las expresiones aritméticas.

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow (F) \mid \text{id}$$

- Esta gramática contiene algunas producciones recursivas por la izquierda. Debemos reescribir dichas producciones.

$$E \rightarrow T E'$$
$$E' \rightarrow + T E' \mid \lambda$$
$$T \rightarrow F T'$$
$$T' \rightarrow * F T' \mid \lambda$$
$$F \rightarrow (E) \mid \text{id}$$


- Hemos comprobado anteriormente que en ocasiones nos encontramos con situaciones en las que no podemos decidir entre varias producciones a elegir.
 - En esos casos, se debe producir retroceso si la entrada no concuerda con el árbol generado por la primera producción elegida.
 - Podemos describir las producciones para retrasar la decisión hasta haber analizado lo suficiente de la entrada como para elegir la opción correcta.

$$a \rightarrow \alpha \beta_1 \mid \alpha \beta_2$$

Se convierte en:

$$a \rightarrow \alpha b$$

$$b \rightarrow \beta_1 \mid \beta_2$$



Recursividad y Factorización

Sea G una GLC y sea P una regla de G

- P es una regla **recursiva** si en la parte derecha aparece el no terminal de su parte izquierda:

$$A \rightarrow xAy \text{ / } x, y \in (\Sigma_T \cup \Sigma_N)^*$$

- P es **recursiva por la izquierda** si es recursiva de la forma

$$A \rightarrow Aw \text{ / } w \in (\Sigma_T \cup \Sigma_N)^*$$



Recursividad y Factorización

Recursividad en más de un paso

- Existe un conjunto de reglas que provocan recursividad:

$$A \rightarrow Bw \ / \ A, B \in \Sigma_N, A \neq B, w \in (\Sigma_T \cup \Sigma_N)^*$$

$$B \rightarrow^* Ax \ / \ x \in (\Sigma_T \cup \Sigma_N)^*$$



Recursividad y Factorización

Una GLC se dice recursiva si cumple una de estas dos afirmaciones:

- Tiene una regla recursiva
- Tiene recursividad en más de un paso



Recursividad y Factorización

Algoritmo para eliminar la recursividad por la izquierda (recursividad en un paso)

Para cada $A \in \Sigma_N$

- Si las producciones de A son de la forma:

$$P: \{ A \rightarrow A \cdot \alpha_1 \mid A \cdot \alpha_2 \mid \dots \mid A \cdot \alpha_n \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_m \}$$

$\alpha_i, \beta_i \in (\Sigma_T \cup \Sigma_N)^*$ y β_i no comienza por A

- Añadir nuevo símbolo: $\Sigma_N = \Sigma_N \cup \{A'\}$

- Sustituir P por:

$$P': \{ \begin{array}{l} A \rightarrow \beta_1 \cdot A' \mid \beta_2 \cdot A' \mid \dots \mid \beta_m \cdot A' \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_m \\ A' \rightarrow \alpha_1 \cdot A' \mid \alpha_2 \cdot A' \mid \dots \mid \alpha_n \cdot A' \mid \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n \end{array} \}$$



Recursividad y Factorización

Algoritmo para eliminar la recursividad por la izquierda en más de un paso

1. Ordenar los símbolos Σ_N : $\{A_1, A_2, \dots, A_n / A_1=S\}$
2. Para $i=1$ hasta n
 1. Para $j=1$ hasta $i-1$
 - Para cada producción $p \in P$ $\{A_i \rightarrow A_j \cdot \beta\}$
 - Si $A_j \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n$
 - Reemplazar p por
 $P': \{ A_i \rightarrow \alpha_1 \cdot \beta \mid \alpha_2 \cdot \beta \mid \dots \mid \alpha_n \cdot \beta \}$
 2. Eliminar recursividad de A_i



Recursividad y Factorización

- Factorización a izquierdas
 - **Problema:** existen producciones de un mismo símbolo no terminal en cuya parte derecha, la primera parte sea común:
$$A \rightarrow \beta \cdot \alpha_1 \mid \beta \cdot \alpha_2 \mid \dots \mid \beta \cdot \alpha_n$$
 - **Solución:** Factorizar a izquierdas mediante el algoritmo para factorizar una GIC



Recursividad y Factorización

Algoritmo para factorizar una GIC

Para cada $A \in \Sigma_N$

- Si las producciones de A son de la forma:

$$P: \{ A \rightarrow \beta \cdot \alpha_1 \mid \beta \cdot \alpha_2 \mid \dots \mid \beta \cdot \alpha_n \}$$

$$\beta, \alpha_i \in (\Sigma_T \cup \Sigma_N)^*$$

- Añadir nuevo símbolo: $\Sigma_N = \Sigma_N \cup \{A'\}$
- Sustituir P por:

$$P': \{ A \rightarrow \beta \cdot A' \}$$

$$A' \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n \}$$



Limpieza o Simplificación de gramáticas

- Se trata de llegar a una forma normal o estándar para las producciones, evitando que la gramática sea inútilmente sencilla o inútilmente profunda.
 - 4 algoritmos
 - 1- Eliminar No Terminales que no deriven cadenas terminales
 - 2- Eliminar Reglas y Producciones que no deriven del axioma
 - 3- Eliminar las λ -producciones
 - 4- Eliminar las producciones unitarias
- NOTA: Los algoritmos se pueden aplicar en distinto orden siempre que se respete 1-2 por un lado (alg. 1 antes que el alg. 2) y 3-4 por otro (alg. 3 antes que el alg. 4)



Limpieza - Algoritmo 1

Eliminar No Terminales que no deriven cadenas terminales

- Sea $G=(\Sigma_T, \Sigma_N, S, P)$ una GIC. Transformar G en $G'=(\Sigma_T, \Sigma_N', S, P')$, tal que $L(G) = L(G')$
 1. Inicializar $\Sigma_N' = \{A \in \Sigma_N / (A \rightarrow w) \in P \text{ y } w \in \Sigma_T^*\}$
 2. Inicializar $P' = \{(A \rightarrow w) \in P / A \in \Sigma_N' \text{ y } w \in \Sigma_T^*\}$
 3. Repetir
 - Añadir No Terminales a Σ_N' :
 - $\Sigma_N' = \Sigma_N' \cup \{A \in \Sigma_N / (A \rightarrow w) \in P \text{ y } w \in (\Sigma_T \cup \Sigma_N')^*\}$

Hasta que no se puedan añadir más elementos No Terminales a Σ_N'



Limpieza - Algoritmo 2

Eliminar Reglas y Producciones que no deriven del axioma.

- Sea $G=(\Sigma_T, \Sigma_N, S, P)$ una GLC. Transformar G en $G'=(\Sigma_T', \Sigma_N', S, P')$, tal que $L(G) = L(G')$
 1. Inicializar $\Sigma_N' = \{S\}$, $P' = \emptyset$, $\Sigma_T' = \emptyset$
 2. Repetir
 - $\forall A \in \Sigma_N' / (A \rightarrow w) \in P$
 - 2.1. Añadir $(A \rightarrow w) \in P'$
 - 2.2. $\forall B \in \Sigma_N / B \in w$: Añadir $B \in \Sigma_N'$
 - 2.3. $\forall a \in \Sigma_T / a \in w$: Añadir $a \in \Sigma_T'$
 - $\Sigma_N' = \Sigma_N' \cup \{A \in \Sigma_N / (A \rightarrow w) \in P \text{ y } w \in (\Sigma_T \cup \Sigma_N')^*\}$
- Hasta que no se puedan añadir nuevas producciones a P'



Limpieza - Algoritmo 3

Eliminar las λ -producciones.

a) Identificar No Terminales Anulables.

Sea $G=(\Sigma_T, \Sigma_N, S, P)$. Extraer conjunto de anulables:

a.1) Inicializar $\eta = \{A \in \Sigma_N / (A \rightarrow \lambda)\}$

a.2) Repetir

Si $B \rightarrow w / w \in (\Sigma_N)^*$ y todos los símbolos $w_i \in \eta$

Añadir $B \in \eta$

Hasta que no se puedan añadir Σ_N a η



Limpieza - Algoritmo 3

b) Eliminar las λ -producciones.

Sustituir producciones $B \rightarrow X_1 X_2 \dots X_n$, eliminando subconjuntos X_i anulables. No incluir $B \rightarrow \lambda$

b.1) Crear $P' = \emptyset$

b.2) Si $B \rightarrow X_1 X_2 \dots X_n \in P$

Entonces Añadir a P' : $B \rightarrow Y_1 Y_2 \dots Y_n$ donde:

- $Y_i = X_i$, si X_i no es anulable
- $Y_i = X_i$ ó λ , si X_i es anulable (combinaciones)
- No introducir $B \rightarrow \lambda$

NOTA: Pueden aparecer nuevas producciones al combinar todas las soluciones $Y_i = X_i$ ó λ

c) Si $S \in \eta \rightarrow$ Añadir $S \rightarrow \lambda$ a P



Limpieza - Algoritmo 4

- Eliminar las producciones unitarias
 - Sea $G=(\Sigma_T, \Sigma_N, S, P)$ una GIC sin λ -transiciones.
Transformar G en $G'=(\Sigma_T, \Sigma_N', S, P')$, tal que $L(G) = L(G')$
 - a) Construir conjuntos unitarios
 $\forall A \in \Sigma_N$ construir:
 $\text{Unitario}(A) = \{B \in \Sigma_N / A \rightarrow *B\}.$
 - b) Eliminar Unitarios:
 1. Inicializar $P' = P$
 2. $\forall A \in \Sigma_N / \text{Unitario}(A) \neq \{A\}$
 $\forall B \in \text{Unitario}(A)$
 $\forall$ producción no unitaria $(B \rightarrow w) \in P$
Añadir $(A \rightarrow w) \in P$
 3. Eliminar todas las producciones no unitarias.



Formas normales para GIC

- Para un mismo LIC existen multitud de GIC
 - Algunas ambiguas
 - Algunas con símbolos inútiles
- Dos formas de Normalizar
 - Forma Normal de Chomsky (FNC)
 - Forma Normal de Greibach (FNG)



Forma Normal de Chomsky (FNC)

- Una gramática $G=(\Sigma_T, \Sigma_N, S, P)$, G.I.C. está en FNC si no contiene λ y todas las producciones son de la forma:

$$A \rightarrow a / A \in \Sigma_N, a \in \Sigma_T$$

ó

$$A \rightarrow BC / A, B, C \in \Sigma_N$$

- Teorema: Si G es una GIC y $\lambda \notin L(G)$, entonces existe G' en FNC / $L(G) = L(G')$



Algoritmo FNC

Sea $G=(\Sigma_T, \Sigma_N, S, P) / \lambda \notin L(G)$

1. Limpiar gramática.

Obtener $G'=(\Sigma_T', \Sigma_N', S, P')$ limpia / $L(G) = L(G')$

Resultado P' : $A \rightarrow w$

si $|w|=1 \rightarrow w \in \Sigma_T$

si $|w|>1 \rightarrow w \in (\Sigma_T \cup \Sigma_N)^+$

2. Quitar Σ_T a la derecha, cuando $|w|>1$

$\forall A \rightarrow w / |w|>1$, donde $A \rightarrow X_1 X_2 \dots X_n$

$\forall X_i = a \in \Sigma_T$, hacer:

Crear símbolo $C_a \in \Sigma_N'$

Transformar $a \in \Sigma_T$ por $C_a \in \Sigma_N$

Crear producción: $C_a \rightarrow a \in P'$

Resultado:

$P': A \rightarrow B_1 B_2 \dots B_n \vee A \rightarrow a$



Algoritmo FNC

3. Eliminar cadenas a la derecha con $|w| > 2$

Si $A \rightarrow B_1 B_2 \dots B_n$

Entonces Transformar producción en:

$A \rightarrow B_1 D_1$

$D_1 \rightarrow B_2 D_2$

...

$D_{n-2} \rightarrow B_{n-1} B_n$

Añadir nuevos no terminales $D_1, D_2, \dots, D_{n-2} \in \Sigma_N$



Forma Normal de Greibach (FNG)

- Una gramática $G=(\Sigma_T, \Sigma_N, S, P)$, G.I.C. está en FNG si no contiene λ y todas las producciones son de la forma:

$$A \rightarrow a\alpha \text{ / } A \in \Sigma_N, a \in \Sigma_T, \alpha \in (\Sigma_T \cup \Sigma_N)^*$$

- FNG alternativa (la que nos gusta):

$$A \rightarrow a\alpha \text{ / } A \in \Sigma_N, a \in \Sigma_T, \alpha \in \Sigma_N^*$$

- Teorema: Si G es una GIC y $\lambda \notin L(G)$, entonces existe G' en FNG / $L(G) = L(G')$



Forma Normal de Greibach (FNG)

Teoremas necesarios:

- Teorema 1: Propagar producciones

Si $A \rightarrow \alpha\beta\gamma$

y $\beta \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$

Entonces Transformar $A \rightarrow \alpha\beta\gamma$ en:

$A \rightarrow \alpha\beta_1\gamma \mid \alpha\beta_2\gamma \mid \dots \mid \alpha\beta_n\gamma$

- Teorema 2: Eliminar recursividad por la izquierda

Sea $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_n \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_m$

Transformar en:

$A \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_m \mid \beta_1 Z \mid \beta_2 Z \mid \dots \mid \beta_m Z$

$Z \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n \mid \alpha_1 Z \mid \alpha_2 Z \mid \dots \mid \alpha_n Z$

Con Z nuevo símbolo no terminal



Algoritmo FNG

Una gramática $G=(\Sigma_T, \Sigma_N, S, P)$, GIC limpia, y supongamos $\Sigma_N=\{A_1, A_2, \dots, A_n\}$ con un orden establecido / $A_1=S$

- Objetivo: Dejar producciones: $A_r \rightarrow A_s \alpha$ / $r < s$
- 1. Repetir $\forall A_j \in \Sigma_N$
 1. Si $A_k \rightarrow A_j \alpha$, con $k > j \rightarrow$ Aplicar Teorema 1
 2. Si $A_k \rightarrow A_j \alpha$, con $k = j$ (recursiva) $\rightarrow$ Aplicar Teorema 2
- Hasta que todas las producciones: $A_r \rightarrow A_s \alpha$ / $r < s$



Forma Normal de Greibach

- Operación para eliminar una producción $A_i \rightarrow A_j\alpha$, con $i < j$
 1. Sacar $A \rightarrow B\alpha$ de P .
 2. $\forall p \in P / B \rightarrow \beta$, añadir $A \rightarrow \beta\alpha$



Forma Normal de Greibach

- Con este proceso, las reglas, si su parte izquierda es un antiguo símbolo no terminal, están en FNG
- Para convertir las reglas con la parte izquierda formada por un nuevo no terminal a FNG, basta con aplicar sobre ellas en Teorema 1



Algoritmo CYK

- Cocke, Younger, Kasami
- Sea $G=(\Sigma_N, \Sigma_T, S, P)$ una GLC, sin producciones compresoras y que está en *Forma Normal de Chomsky*.
- Sea $w \in \Sigma_T^*$
- **Objetivo:** Determinar si $w \in L(G)$



Algoritmo CYK

- **Definición:** $w_{ij} \in \Sigma_T^*$ es una subcadena de $w \in \Sigma_T^*$:

- comienza con el símbolo de la posición i
- Longitud j

$$w_{ij} = w_i w_{i+1} \dots w_{j-1}$$

Observación: $w_{1n} = w$ / $|w| = n$

- **Definición:** $N_{ij} = \{ A \in \Sigma_N \mid A \xrightarrow{*} w_{ij} \}$



Algoritmo de Cocke-Younger-Kasami (CYK)

Este algoritmo se aplica a gramáticas libres de contexto en **forma normal de Chomsky**.
Supongamos que n es la longitud de la palabra.

1. Para $i=1$ hasta n
 1. $N_{i1} = \{A \mid A \rightarrow a \text{ y el símbolo } i\text{-ésimo de } w \text{ es } a\}$.
2. Para $j=2$ hasta n
 1. Para $i=1$ hasta $n-j+1$
 1. $N_{ij} = \emptyset$
 2. Para $k=1$ hasta $j-1$
 1. $N_{ij} = N_{ij} \cup \{A \mid A \rightarrow BC, \quad \begin{array}{l} B \in N_{ik} \\ C \in N_{i+k+1, j-k} \end{array} \}$
3. Si $S \in N_{1n} \rightarrow w \in L(G)$



Algoritmo de Cocke-Younger-Kasami (CYK)

- Este algoritmo calcula para todo i, j ($i \in \{1, \dots, n\}$, $j \leq n - j + 1$), el conjunto de variables que generan w_{ij} , donde w_{ij} es la subcadena de w que comienza en el símbolo que ocupa la posición i y que contiene j símbolos.
- La palabra u será generada por la gramática si la variable inicial S pertenece al conjunto N_{1n} .
- Los cálculos se pueden organizar en una tabla.



Algoritmo de Cocke-Younger-Kasami

	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$
$i=1$	N_{11}	N_{12}	N_{13}	N_{14}	N_{15}
$i=2$	N_{21}	N_{22}	N_{23}	N_{24}	
$i=3$	N_{31}	N_{32}	N_{33}		
$i=4$	N_{41}	N_{42}			
$i=5$	N_{51}				

